Efficient deep neural network computing by Tensor Train operations

Liancheng Fang

Department of Mathematics

1th Dec 2019

Tensor Train decomposition

- Definition:

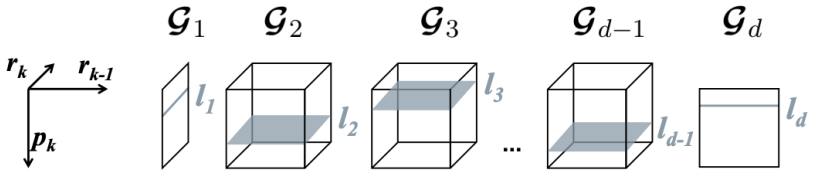

Figure 1: Tensor-Train Factorization Model: To reconstruct one entry in the target tensor, one performs a sequence of vector-matrix-vector multiplications, yielding a scalar.

(Yinchong Yang, 2017)

- TT property:
  (flexibility:) Given a TT rank bound r, the best approximation to $\mathcal{T}$ always exists, denote by $\mathcal{T}^{\text{best}}$
  (quasi-optimal:) TT-SVD algorithm gives us approximation $\mathcal{A}$ s.t

$$\|\mathcal{T} - \mathcal{A}\|_F \leq \sqrt{d-1}\|\mathcal{T} - \mathcal{T}^{\text{best}}\|_F$$

Use TT in deep neural network: Motivation

- (Tensorizing neural network. Alexander Novikov et al 2015)
- In a MLP, they initilize the weight matrix as a tensor in TT format, which makes the matrix-vector multiplication in forward pass becomes TT-matrix-dense-vector multiplication.
- TT-Dense multiplication: $\mathcal{O}(4r^3(M+N)), M = \prod_{i=1}^{d} m_i, N = \prod_{i=1}^{d} n_i$
- Dense-Dense multiplication: $\mathcal{O}(MN)$

Use TT in deep neural network: one step further

Let $X_1, X_2, \ldots, X_n$ be a sequence of independent and identically distributed random variables with $\mathrm{E}[X_i] = \mu$ and $\mathrm{Var}[X_i] = \sigma^2 < \infty$, and let

$$S_n = \frac{X_1 + X_2 + \cdots + X_n}{n} = \frac{1}{n}\sum_i^n X_i$$

denote their mean. Then as n approaches infinity, the random variables $\sqrt{n}(S_n - \mu)$ converge in distribution to a normal $\mathcal{N}(0, \sigma^2)$.

Main idea: If we convert input x to its TT format, the time complexity of matrix-vector multiplication in deep neural network is reduced to $\mathcal{O}(dn^2r^4)$ To make the idea valid, we hope

- Convert input to TT is a good feature extraction.
- We can efficiently compute some non-linear function of TT, e.g $\mathrm{ReLU}(\mathcal{T})$, where $\mathcal{T}$ is a tensor in TT-format.

Problem: Feature extraction by TT-SVD

Unfortunately, the feature extracted by TT-SVD algorithm seems to be very bad!

t-SNE:

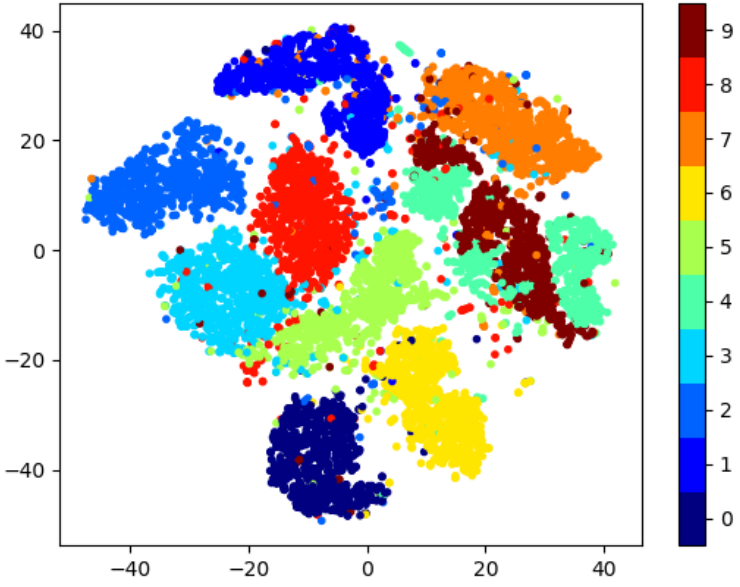

Figure 1: raw-tSNE

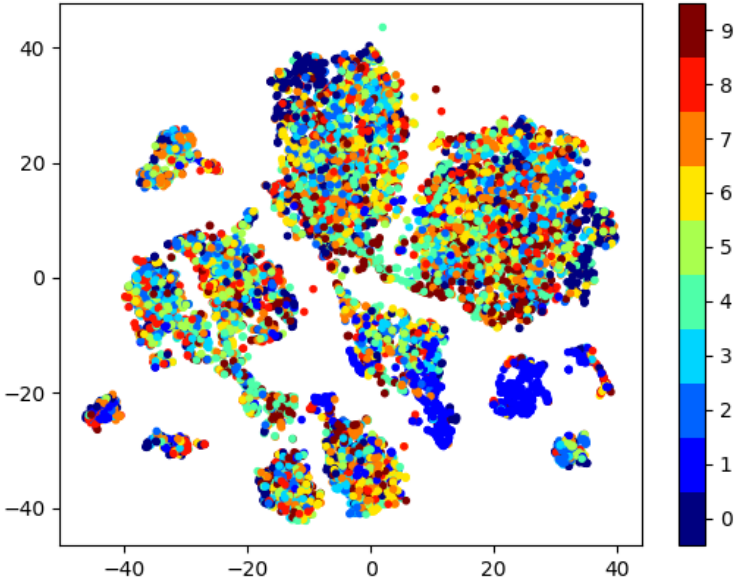

Figure 2: TT-tSNE

Problem: Feature extraction by TT-SVD

- Wide Resnet is able to fit complete noise pixels perfectly(training error=0)
  (Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, Oriol Vinyals. Understanding deep learning requires rethinking generalization. 2017)
- but Wide Resnet fails to fit TT features of MINIST/CIFAR10

Non-linear function of TT

If we put the problem of bad features aside, we can develop some alogorithms to approximately compute non-linear function of TT tensor.

- $\|\mathcal{T}\|_\infty := \max_{\underline{i}=(i_1,\ldots,i_d)} |\mathcal{T}_{\underline{i}}|$
- Point-wise inverse: $(\mathcal{T})^{-1}_{\underline{i}=(i_1,\ldots,i_d)} = \frac{1}{u_{\underline{i}}}$
- $$(sign(\mathcal{T}))_{\underline{i}} := \begin{cases} 1 & \mathcal{T}_{\underline{i}} > 0 \\ -1 & \mathcal{T}_{\underline{i}} < 0 \\ \pm 1 & \mathcal{T}_{\underline{i}} = 0 \end{cases}$$
- $\mathrm{ReLU}(\mathcal{T}) = \mathcal{X}_I(u) = \frac{1}{2}(1 - \mathrm{sign}(a1 - u)) \odot \mathcal{T}, a \in \mathbb{R}$ (if we can compute sign, we can compute ReLU)

Newton method: detail

- Power iteration for max norm
- Newton method to solve equation: $f(x) = \frac{1}{x} - \mathcal{T}_{\underline{i}}$, we get iteration procedure

$$x_{n+1} = 2x_n - \mathcal{T}_{\underline{i}} x_n^2$$

  This algorithm converge quadratically **when** $\|1 - \mathcal{T} \odot \mathcal{T}_0\|_\infty < 1$, where $\mathcal{T}_0$ is an initial guess.
- Newton method to solve equation: $f(x) = x^2 - 1$, which gives us iteration procedure: $x_{n+1} = \frac{1}{2}\left(x_n + \frac{1}{x_n}\right)$, we can show this procedure will always converge, but **no guarantee about convergent rate**. (or apply gradient descent to objective function: $\|x^2 - 1\|_F^2$)
- **Problem:** No bound for iteration number. Empirically, not converge sometimes.

Optimization method

We have shown that if we can compute $sign(\mathcal{T})$, we can compute $ReLU(\mathcal{T})$.

- Empirical result shows that point-wise inverse is slow(involveiterative alg in iterative alg) and not stable, so we think about avoiding compute inverse. Notice that finding zero point of $x^2 - 1$ equivalent to minimize $\|x^2 - 1\|_F^2$.
- We tried vanilla GD and Riemannian GD.
- Need to do rouding after every iteration!
- Experiment: not converge, and expensive

Optimization method: change objective function

- Actually one iteration is enough, if we use the right objective function! (Deep Layers as Stochastic Solvers. Adel Bibi, et al 2018)
- $\mathbf{x}^l$: input of layer l.
- Let $F\left(\mathrm{x}^l\right) = \frac{1}{2}\left\|\mathbf{A}^\top \mathbf{x}^l - \mathbf{x}^{l-1}\right\|^2 - \mathbf{b}^\top \mathbf{x}^l$
- $\mathbf{a} = \frac{1}{L}\left(\mathbf{A}\mathbf{x}^{l-1} + \mathbf{b}\right)$, $g_{\mathbf{a}}(\mathbf{x}) = \left(\mathbf{a}^2 + \epsilon\right) 1_{\mathbf{x} \leq 0}$, ϵ small positive number.
- **Claim:** Apply **one** iteration of **Proximal gradient descent** to $F\left(\mathrm{x}^l\right) + g_{\mathrm{a}}\left(\mathrm{x}^l\right)$ is equivalent to one forward of ReLU block:

$$\mathbf{x}^l \leftarrow \mathrm{Re}\,LU\left(A\mathbf{x}^{l-1} + b\right)$$

if $\mathbf{x}^l$ is initilized as 0.

Forward pass as optimization solver

- If we apply vanilla GD to $F(x) + g(x)$?
- $\mathbf{x}^l \leftarrow \mathbf{x}^l - \frac{1}{L}\nabla\left(F\left(\mathbf{x}^l\right) + g_{\mathbf{a}}\left(\mathbf{x}^l\right)\right)$
- Plug in expressions for $F(x), g(x)$, we get

$$\mathbf{x}^l \leftarrow \left(\mathbf{I} - \frac{1}{L}\mathbf{A}\mathbf{A}^T\right)\mathbf{x}^l + \frac{1}{L}\left(\mathbf{A}\mathbf{x}^{l-1} + \mathbf{b}\right) + \delta\left(\mathbf{x}^l\right)$$

- We further approximate δ function by

$$\mathbf{x}^l \leftarrow \begin{cases} \left(\mathbf{I} - \frac{1}{L}\mathbf{A}\mathbf{A}^\top\right)\mathbf{x}^l + \frac{1}{L}\left(\mathbf{A}\mathbf{x}^{l-1} + \mathbf{b}\right) & \text{if } \mathbf{x}^l \neq 0 \\ \left(\mathbf{I} - \frac{1}{L}\mathbf{A}\mathbf{A}^\top\right)\mathbf{x}^l + \frac{1}{L}\left(\mathbf{A}\mathbf{x}^{l-1} + \mathbf{b} + S\right) & \text{if } \mathbf{x}^l = 0 \end{cases}$$

- Now we can compute this forward pass by basic TT operations, and hope this will approximate ReLU block.

Summary: Compute ReLU of TT

- General idea: use optimization method, which only envolove basic TT operation.
- 1. Newton method(two much iteration)
- 2. GD to $\|x^2 - 1\|_F^2$ (not converging, rounding is expensive)
- 3. One iteration of GD to $F(x) + g_a(x)$
- Experiment: VGG-19-bn, Imagenet: top1 acc:70.258%, top5 acc:89.244%
  Baseline: top1: 74.24%, top5:91.85%

Convolution for TT input

- Since TT with fixed rank forms a Riemannian manifold, given a field of tensors in TT format $\{\mathcal{X}_i\}_{i=1}^{N}$ and kernel of size K, defined by $\{w_{ij}\}_{i=1,j=1}^{N,nd}$, we can define the convolution operator as:

$$\{\mathcal{X}_i\} \star \{w_{ij}\} = \arg\min_{\mathcal{M}} \sum_{i=1}^{N} d^2\left(\mathcal{X}_i, \mathcal{M}, \{w_{ij}\}\right)$$

- (ManifoldNet: A Deep Network Framework for Manifold-valued Data. Rudrasis *et al* 2018)

Thanks!